

1 Параллелизм данных (DP)

1.1 Распределенный параллелизм данных (DDP)

Самый простой способ организации параллелизма вычислений на GPU — это разбить батч данных между GPU. Такая стратегия называется **параллелизмом данных**.

Каждая GPU получает свою часть батча и выполняет прямое и обратное распространение независимо друг от друга. А параметры модели копируются (реплицируются) между девайсами.

На каждом ранге выполняется вычисление локального лосса по своим объектам:

$$\mathcal{L}_i^{(t)} = \mathcal{L}_{w_t}(B_i)$$

Каждый девайс выполняет обратное распространение в конкретном слое модели на основе её входа и на основе своего лосса. Это значит, что градиенты на каждом ранге будут разные. А веса — одни.

$$g_i^{(t)} = \nabla_{w_t} \mathcal{L}_i^{(t)}$$

Чтобы все копии модели оставались одинаковыми, нужно синхронизировать градиенты. Для этого применяется **AllReduce** по градиентам:

$$g^{(t)} = \frac{1}{N} \sum_{j=1}^N g_j^{(t)}$$

После этого на каждом ранге будет лежать одинаковый усредненный градиент, поэтому мы можем на каждом девайсе независимо выполнить шаг оптимизатора.

Как видно, главная коммуникация в DDP — это **AllReduce**. Размер передаваемых данных примерно равен размеру градиентов модели. При обучении больших моделей это может занимать много времени.

Поэтому на практике DDP не ждет завершения всего обратного распространения, чтобы начать коммуникацию. Градиенты ведь считаются последовательно по слоям. Таким образом, мы можем вычислить градиенты одного трансформерного слоя и выполнять **AllReduce** над ними, а в это время асинхронно начать вычислять градиенты следующего трансформерного слоя. Таким образом мы можем прятать коммуникации под вычислениями.

Это называется **bucketing**. Мы не ждём вычисления всех градиентов. Как только мы вычисляем достаточно градиентов, что их размер превышает заданный нами размер бакета, мы начинаем коммуникацию.

Размер бакета не обязательно равен размеру вектора градиентов одного трансформерного слоя, как было в примере. Это настраиваемый гиперпараметр, поскольку большие бакеты приводят к более высокой пропускной способности, но увеличивают задержку из-за более поздней коммуникации.



Рис. 1: Схема распределенного параллелизма данных между двумя нодами.

Малые бакеты наоборот, снижают пропускную способность, зато коммуникации стартуют раньше.

Для моделей с большими слоями полезно использовать большие бакеты, а для малых — меньшие.

1.2 ZeRO

DDP хорошо масштабирует вычисления, но плохо масштабирует память. На каждом ранге нам приходится хранить активации и реплики параметров модели, градиентов и состояний оптимизатора.

При использовании оптимизаторов, таких как, например, Adam или AdamW, для одного параметра в **fp32** необходимо хранить 16 байт данных: 4 байта на сам параметра, 4 байта на его градиента, 8 байта на состояние оптимизатора. Это сильно раздувает потребление памяти. Таким образом, хоть мы и выигрываем от увеличения вычислительных мощностей, мы почти не получаем выигрыша от увеличения памяти GPU, поскольку храним реплики.

Эту проблему решает **оптимизация нулевой избыточности (ZeRO, Zero Redudancy Optimizer)**. Основная идея заключается в том, чтобы не реплицировать градиенты, состояния оптимизатора и параметры моде-

ли между всеми рангами, а шардировать их (разделять на равные части) между всеми рангами и собирать по мере необходимости.

Существует три стадии ZeRO. Рассмотрим первые две из них.

ZeRO-1 В этой стадии мы шардируем только состояния оптимизатора. Параметры и градиенты всё ещё хранятся на своих рангах.

После обратного распространения градиенты синхронизируются как в DDP с помощью `AllReduce`. Затем каждый ранг обновляет только свою часть состояния оптимизатора и соответствующую ей часть параметров. После этого обновленные параметры нужно снова сделать доступными всем рангам с помощью `AllGather`.

ZeRO-2 В ZeRO-2 шардируются и состояния оптимизатора, и градиенты. Теперь после обратного распространения слоя нам не нужно хранить полную копию всех градиентов на каждом ранге

$$g_i^{(t)} = \nabla_{w_i} \mathcal{L}_i^{(t)}$$

Вместо этого мы шардируем градиенты между рангами

$$g_i^{(t)} = [g_{i1}^{(t)}, g_{i2}^{(t)}, \dots, g_{iN}^{(t)}]$$

Ранг i хранит только $g_{ii}^{(t)}$. Вместо `AllReduce` нужно использовать `Reduce-Scatter` после вычисления обратного распространения каждого слоя. Так каждый ранг i получит все градиенты $\{g_{ji}^{(t)}\}_{j=1}^N$, соответствующие его шардированной части и выполнит редукцию только по ней:

$$g_i^{(t)} = \frac{1}{N} \sum_{j=1}^N g_{ij}^{(t)}$$

1.3 FSDP

ZeRO-3 требует особого внимания. Эта стадия также называется **полностью шардированным параллелизмом данных (fully sharded data parallel, FSDP)**. Как понятно из названия, FSDP шардирует всё: и градиенты, и состояния оптимизатора, и параметры.

Если мы используем N рангов, то в идеальном случае можем уменьшить потребление памяти в N раз: в N раз уменьшается объем активаций на девайсе за счёт уменьшения количества объектов в батче и в N раз уменьшается объем хранимых параметров, градиентов и состояний оптимизатора за счёт шардирования.

Это самая агрессивная стадия шардирования и она требует большого количества постоянных коммуникаций всех рангов в группе DP-параллелизма.

Если параметры зашардированы, то перед вычислением прямого пространства слоя ранг должен собрать все шарды со всех остальных рангов с помощью **AllGather**. Чтобы не раздувать потребление памяти репликами параметров, после завершения вычисления слоя мы освобождаем память тех параметров, которые не являются шардом на текущем ранге.

В обратном распространении нам также необходимы все параметры слоя, поэтому мы также вынуждены сначала собрать их с помощью **AllGather**. После вычисления обратного распространения, вычисленные градиенты нужно разложить по рангам с помощью **ReduceScatter**, как мы делали в ZeRO-2. После вычисления слоя, нужно освободить память шардов, не относящихся к текущему рангу.

Выполнение **AllGather** коммуникации для сбора следующего слоя мы можем прятать под вычисления текущего слоя. Аналогично и с **ReduceScatter**.

Скрытие стрима коммуникаций под стримом вычислений — основная оптимизация FSDP. Степень скрытия коммуникаций оценивает метрика communication-computation overlap:

$$\text{Overlap}_{\text{comm-comp}} = \frac{T_{\text{comp}}}{T_{\text{comp}} + T_{\text{idle}}},$$

где T_{idle} означает время, которое ранг находится в простое из-за ожидания завершения коммуникации.

Дальнейшие оптимизации FSDP уменьшают время коммуникации за счёт снижения накладных расходов на реаллокацию памяти, изменения методов шардирования и использования двойной буферизации. Это — тонкие инженерные детали, которые мы не будем рассматривать.

2 Тензорный параллелизм (TP)

FSDP уменьшает потребление памяти и ускоряет вычисления, разделяя батч на части.

Однако, мы не можем ускорить таким образом вычисление слоя, обрабатывающего один объект в батче. Мы всё ещё можем упереться в хранение активаций.

Попробуем параллелизовать вычисления внутри одного слоя, разделяя вычисления самих тензоров между разными рангами. Такой метод параллелизма называется **тензорным параллелизмом (TP)**

2.1 Параллелизм по столбцам и строкам

Тензорный параллелизм использует математические свойства матричного умножения. Во-первых:

$$A \cdot B = A \cdot \begin{bmatrix} B_1 & B_2 & \dots \end{bmatrix} = \begin{bmatrix} A \cdot B_1 & A \cdot B_2 & \dots \end{bmatrix}$$

И во-вторых:

$$A \cdot B = \begin{bmatrix} A_1 & A_2 & \dots \end{bmatrix} \cdot \begin{bmatrix} B_1 \\ B_2 \\ \dots \end{bmatrix} = \sum_{i=1} A_i B_i$$

Первый способ позволяет организовать **параллелизм по столбцам**: каждый ранг держит полную матрицу A и несколько столбцов матрицы B . Этот ранг вычисляет $A \cdot B_i$.

Например, при вычислении линейного слоя, это позволяет разделять веса между рангами

$$Y_i = XW_i,$$

увеличив скорость вычисления этого слоя. Однако параллелизм по столбцам не уменьшает объем хранимых активаций в памяти.

Попробуем воспользоваться вторым свойством матричного умножения, которое мы рассматривали ранее: разрежем матрицу X по столбцам, а матрицу W — по строкам.

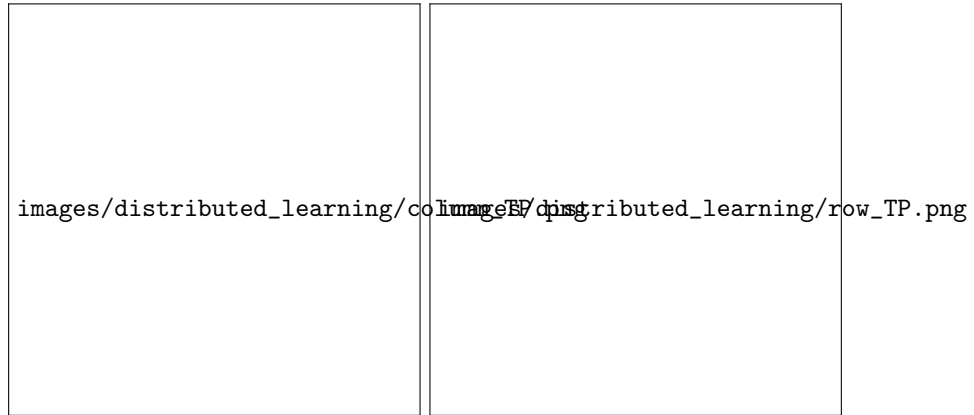


Рис. 2: Схема параллелизма по столбцам (слева) и по строкам (справа). Взято из Ultra-Scale Playbook

Этот метод называется **параллелизмом по строкам**. Каждый девайс хранит лишь часть активаций и часть весов. Однако взамен мы платим очень долгими и тяжелыми **AllReduce** коммуникациями после каждого слоя, необходимыми для вычисления суммы

$$Y = \sum_{i=1} X_i W_i$$

При вычислении FFN-блока мы можем избежать лишних коммуникаций, вычисляя первый линейный слой с помощью параллелизма по столбцам, а следующий — с помощью параллелизма по строкам.



Рис. 3: Схема вычисления FFN-блока с использованием сначала параллелизма по столбцам, а следом — параллелизма по строкам. Взято из Ultra-Scale Playbook.

Поскольку внутренняя размерность FFN-блока обычно больше, чем входная и выходящая, то такой подход позволяет также значительно выиграть в хранении активаций.

Теперь рассмотрим блок многоглавого внимания. Головы внимания вычисляют результат независимо друг от друга. А это значит, мы можем попробовать вычислять их параллельно на разных девайсах.

Рассмотрим на примере двух голов внимания и двух девайсов. Пусть на одном девайсе — одна тройка матриц Q_1 , K_1 и V_1 , а на втором девайсе другая тройка Q_2 , K_2 и V_2 .

Вычисление выхода механизма внимания одной головы тогда будет происходить также, как при использовании параллелизма по столбцам. Важно: при этом мы никак не ломаем кернел FlashAttention, на каждом девайсе мы просто запускаем кернел над разными данными.

После вычисления результата голов O_1 и O_2 , их необходимо сконкатенировать и спроецировать с помощью матрицы W_O :

$$O = [O_1 \quad O_2] W_O$$



Рис. 4: Применение тензорного параллелизма при вычислении многоглавого внимания. Взято из Ultra-Scale Playbook.

Разобьём матрицу W_O по строкам. Пусть W_{O_1} лежит на первом девайсе, а W_{O_2} — на втором. Тогда

$$O = \begin{bmatrix} O_1 & O_2 \end{bmatrix} \cdot \begin{bmatrix} W_{O_1} \\ W_{O_2} \end{bmatrix}$$

Это значит, что каждый ранг сначала первый ранг может вычислить $O_1 W_{O_1}$, а второй ранг — $O_2 W_{O_2}$, после чего каждый из рангов может получить результат

$$O = \sum_{i=1}^2 O_i W_{O_i}$$

с помощью AllReduce.

2.2 Параллелизм последовательностей (SP)

Мы применили параллелизм к FFN-блоку и к механизму внимания. Однако, ещё остались Dropout и LayerNorm слои. Поскольку преобразования

токенов внутри этих операций применяются независимо друг от друга, мы можем разделить последовательность между различными девайсами.

Такой способ параллелизма называется **параллелизмом последовательностей (SP)**. Параллелизм последовательностей также позволяет сократить объем хранимых активаций на одном ранге.

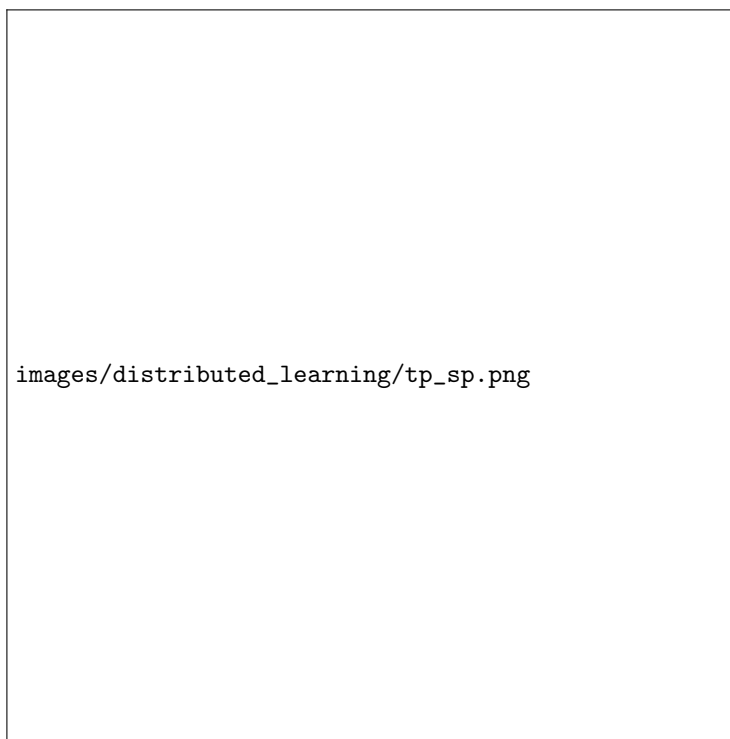


Рис. 5: TP+SP параллелизм. Взято из Ultra-Scale Playbook.

Параллелизм последовательностей идет в паре с тензорным параллелизмом. Чтобы не было простоя GPU, важно чтобы

$$\text{rank}(\text{TP}) = \text{rank}(\text{SP})$$

После вычисления, например, нормализации, необходимо вычислять механизм внимания. Для вычисления механизма внимания с тензорным параллелизмом, необходимо с помощью **AllGather** собрать всю последовательность.

Тензорный параллелизм в паре с параллелизмом последовательностей — мощный инструмент параллелизма вычислений, который позволяет уменьшить объем хранимых активаций и при этом увеличить скорость вычислений слоя. Но из-за тяжелых **AllReduce** и **AllGather**-коммуникаций, которые не получается скрыть под стримом вычислений, необходимо держать коммуникации строго внутри одной ноды.

3 Контекстный параллелизм (CP)

При использовании тензорного параллелизма в механизме внимания нам необходимо на каждом девайсе собирать всю последовательность и хранить активации по ней. Это может привести к нехватки памяти при обучении больших моделей на длинном контексте.

Для решения этой проблемы применяется **контекстный параллелизм (CP)**. Также как и SP, контекстный параллелизм делит последовательность между рангами, при этом сохраняя возможность вычислять внимание по всему контексту.

3.1 Кольцевое внимание

Для реализации вычисления внимания по всему контексту при разделении последовательности между девайсами используется механизм **кольцевого внимания**.

Каждый ранг хранит свой блок запросов Q_i и постепенно получает блоки K и V от других рангов по кольцу GPU.

Пусть есть N рангов. Разобьём последовательность на N блоков:

$$X = [X_1, X_2, \dots, X_N]$$

На ранге i хранятся локальные Q_i , K_i , V_i . Чтобы вычислить O_i , рангу i нужно применить внимание между Q_i и всеми блоками $\{(K_i, V_i)\}_{i=1}^N$.

Вместо того, чтобы сразу собирать все K и V на каждом ранге, блоки передаются по кольцу. На первом этапе так:

$$\text{Rank}_1 \xrightarrow{(K_1, V_1)} \text{Rank}_2 \xrightarrow{(K_2, V_2)} \dots \xrightarrow{(K_{N-1}, V_{N-1})} \text{Rank}_N \xrightarrow{(K_N, V_N)} \text{Rank}_1$$

На следующем этапе:

$$\text{Rank}_1 \xrightarrow{(K_N, V_N)} \text{Rank}_2 \xrightarrow{(K_1, V_1)} \dots \xrightarrow{(K_{N-2}, V_{N-2})} \text{Rank}_N \xrightarrow{(K_{N-1}, V_{N-1})} \text{Rank}_1$$

И так $N - 1$ раз.

Однако, в этом случае на одном ранге мы вычисляем лишь часть матрицы S . Пусть, например, на одном ранге на текущем шаге лежат матрицы Q_i , K_j и V_j . Тогда мы сможем вычислить лишь блок

$$S_{i,j} = \frac{Q_i K_j^T}{\sqrt{d}}$$

по которому даже не посчитать статистики Softmax, ведь мы сможем это сделать лишь после того, как посчитаем все блоки $\{S_{i,j}\}_{j=1}^N$ для каждой

пары (K_j, V_j) . Получается, связать вместе FlashAttention и контекстный параллелизм невозможно?

На каждом шаге кольцевого внимания мы не вычисляем конечный результат FlashAttention. Мы каждый раз обновляем статистики глобальные m_t , l_t и выход O_t . Мы получим корректное значение O_t лишь на последнем шаге.

3.2 Зиг-заг кольцевое внимание

У кольцевого внимания есть еще одна проблема. Обычно, в LLM, мы считаем не просто внимание, а маскированное внимание. В таком внимании мы применяем треугольную маску.

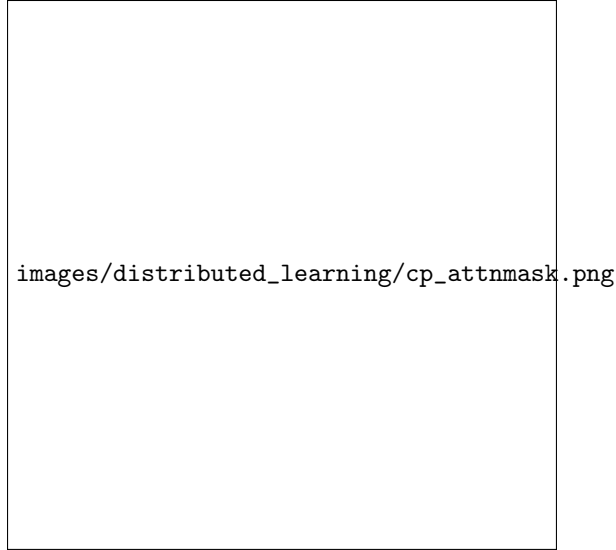


Рис. 6: Разделение ‘зон ответственности’ вычисления матрицы внимания между рангами при контекстном параллелизме. Взято из Ultra-Scale Playbook.

Из-за этого на некоторых рангах лежат блоки Q_i , для которых мы пропускаем вычисление большей части блоков $S_{i,j}$, поскольку они полностью замаскированы.

Из-за этого вычисления становятся несбалансированными между рангами, что приводит к простоем некоторых GPU.

Чтобы это исправить мы можем изменить порядок распределения токенов между рангами так, чтобы каждая нагрузка была равномерно распределена между всеми рангами.

Такой подход называется **зиг-заг кольцевым вниманием (Zig-Zag Ring Attention)**

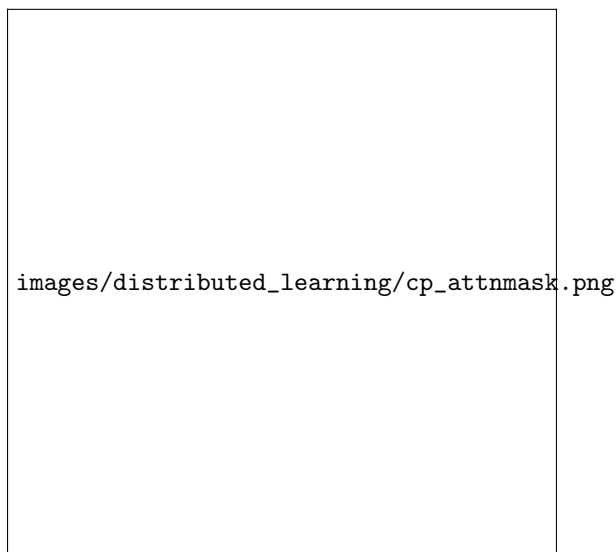


Рис. 7: Распределяем токены так, чтобы выровнять нагрузку на ранги. Взято из Ultra-Scale Playbook.

4 Конвейерный параллелизм (PP)

4.1 Зачем нужен PP

При использовании тензорного параллелизма мы ограничены в масштабировании размерами одной ноды.

Мы также не можем сильно масштабироваться с использованием CP, поскольку при росте CP увеличивается время вычисления механизма внимания из-за увеличения количества этапов.

Допустим, мы используем $TP = 8$ и $CP = 8$ степень параллелизма. Это позволит распараллелить наши вычисления на 64 GPU.

Мы можем существенно нарастить параллелизм, используя FSDP. При $DP = 512$, $TP = 8$ и $CP = 8$ мы могли бы параллелизовать вычисления на 32768 картах.

Казалось бы, это успех. Если потребуется больше видеокарт, то мы просто будем наращивать степень DP. Но не всё так просто.

TP уже занимает каждую ноду целиком. CP и DP приходится полагаться на более медленную межнодовую связь. Для CP требуется выполнять коммуникации внутри механизма внимания, поэтому желательно, чтобы ноды рангов CP находились рядом.

Получается, что для коммуникаций DP-рангов остаются только самые длинные маршруты. А чем длиннее маршрут коммуникации, тем ниже пропускная способность.

Это особенно сильно сказывается на ZeRO-3 (FSDP), где для вычисле-

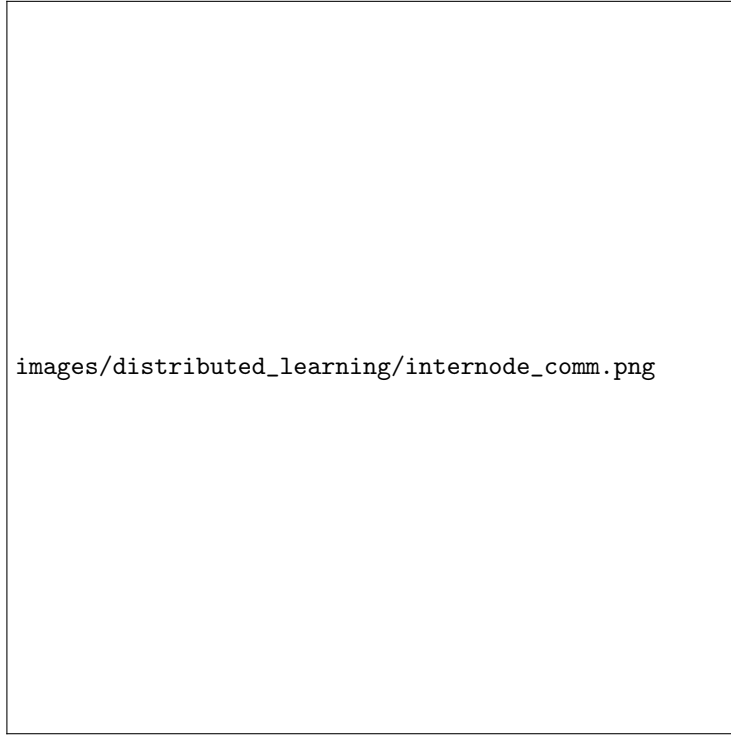


Рис. 8: Пропускная способность межндовой коммуникации в зависимости от количества нод. Взято из Ultra-Scale Playbook.

ния прямого прохода каждого слоя необходимо выполнить **AllGather**, а для обратного распространения — **AllGather** и **ReduceScatter**.

Использовать ZeRO-2 и ZeRO-3 степени шардирования при таком масштабировании становится слишком дорого по коммуникациям. А при использовании ZeRO-1 из-за реплицирования параметров и градиентов мы довольно скоро упрёмся в ограничения по объёму памяти.

Решением будет добавить новую степень параллелизма — **конвейерный параллелизм**.

4.2 Стадии конвейерного параллелизма

Конвейерный параллелизм делит модель не в ‘ширину’, как FSDP, а в ‘глубину’.

Пусть модель состоит из последовательности слоёв:

$$f(x) = f_L(f_{L-1}(\dots f_2(f_1(x))))$$

Конвейерный параллелизм разбивает эти слои между несколькими рангами. Например, если есть P **стадий конвейера (pipeline stages)**, то мо-

дель можно представить так:

$$f(x) = s_P(s_{P-1}(\dots s_2(s_1(x)))) ,$$

где s_i — блок слоёв, который находится на i -й стадии конвейера. Каждая стадия хранит параметры и градиенты только своих слоёв, но целыми, не разнесенными между рангами, тензорами.

4.3 AFAB

При прямом распространении, первая стадия получает входной батч и считает свою часть модели. После этого она передаёт активации следующей стадии.

Коммуникации между стадиями конвейерного параллелизма обычно — point-to-point операции, то есть **send** и **recv**.

Во время обратного распространения направление коммуникаций меняется. Градиент по входу стадии передаётся назад на предыдущую стадию.

Механизм напоминает конвейер, отсюда и название. Однако, для эффективной работы конвейера необходимо настроить расписание обработки данных различными стадиями.

Самое простое расписание — **AFAB** (All Forward, All Backward). Идея проста: сначала выполняем прямое распространение для всех стадий, а потом — обратное распространение в обратном порядке.

AFAB — крайне неэффективное расписание. Чем больше стадий, тем больше простой каждого ранга. Другая важная проблема: необходимо хранить полные активации всех батчей вплоть до старта обратного распространения.

Отправлять весь батч на пайплайн — достаточно наивное решение. Разделим батч на части — **микробатчи**.

В AFAB мы выполняем стадии последовательно для всего батча. Попробуем обрабатывать стадиями не батч целиком, а микробатч. Как только первый ранг закончит обработку первого микробатча, он одновременно передаёт полученные активации следующей стадии и начинает обрабатывать второй микробатч.

Чем больше микробатчей — тем меньше простой. Поэтому для конвейерного параллелизма выгодно делать микробатч размером в один объект.

4.4 1F1B

Более эффективное расписание, позволяющее эффективнее загрузить ранг — **1F1B** (**1 Forward 1 Backward**). Его идея заключается в том, что мы можем начать вычислять обратное распространения не дожидаясь окончания прямого распространения на всех рангах.

1F1B позволяет начать вычисления обратного распространения раньше, что позволяет хранить меньше активаций.



Рис. 9: AFAB. Взято из Ultra-Scale Playbook.

Для дальнейшего уменьшения простоя рангов и уменьшения хранимых активаций, можно разбить каждую физическую стадию на несколько виртуальных, строить более эффективные расписания прямого и обратного распространения и прятать коммуникации под вычисления.

Эти идеи реализуют **interleaved pipeline**, **DualPipe pipeline** и **ZeroBubble pipeline**. Но мы не будем их рассматривать.

5 Иерархия параллелизмов

Как мы уже обсуждали, разные параллелизмы имеют разные преимущества и недостатки. Попробуем расположить все параллелизмы в одной иерархии.

Тензорный параллелизм должен располагаться строго внутри одной ноды, поэтому возьмем $TP = 8$.

В контекстном параллелизме каждый этап кольцевого внимания может вычисляться с использованием тензорного параллелизма. Пусть $CP = 8$. Тогда мы уже можем увеличить общую степень параллелизма до $TP \cdot CP = 64$.

Чтобы не было простоя GPU при вычислении нормализации и слоёв

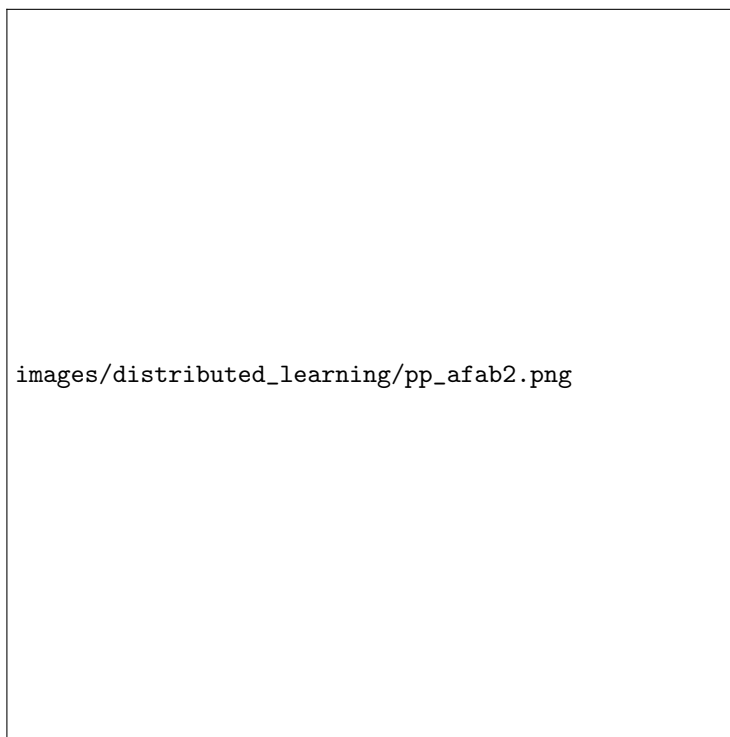


Рис. 10: AFAB по микробатчам. Взято из Ultra-Scale Playbook.

Dropout, необходимо использовать параллелизм последовательностей, причем $SP = TP \cdot CP = 64$.

Далее — вычисление FFN-блока. Мы рассматривали как его вычислять с помощью тензорного параллелизма, но его степень ограничена 8. Чтобы решить эту проблему, используется экспертный параллелизм, который, однако, мы можем применять только для параллельного вычисления специальных МоЕ-блоков, которые мы рассмотрим позднее.

На данный момент в распределенном обучении участвует 64 GPU. Применим конвейерный параллелизм. Объединим несколько слоёв модели в одну стадию конвейера так, чтобы всего было 16 стадий. Тогда $PP = 16$.

В вычислении каждой стадии участвует 64 GPU, а это значит, что общее число GPU в нашем распределенном обучении достигло $TP \cdot CP \cdot PP = 1024$.

Добить нужную степень параллелизма мы можем с помощью DP с ZeRO-1 шардированием. При $DP = 32$ мы достигнем $TP \cdot CP \cdot PP \cdot DP = 32768$ GPU.

Такое количество GPU открывает дорогу к обучению по настоящему больших сетей с триллионами параметров. Но перед этим необходимо подготовить архитектуру модели.



Рис. 11: 1F1B. Взято из Ultra-Scale Playbook.